Benchmarking de MongoDB frente a PostgreSQL JSONB bajo contención de CPU y memoria

Alix Montesinos
Bases de Datos en la Nube · IT2iD
Universidad Politécnica de Chiapas

Noviembre de 2026

Abstract

Se comparan MongoDB y PostgreSQL con `JSONB` sobre el mismo conjunto de 500000 documentos sintéticos, con dos consultas de agregación y tres perfiles de recursos aplicados con *cgroups* v2, a tres niveles de concurrencia y con tres repeticiones por celda: 108 mediciones en 36 celdas. El experimento mide lo que ocurre de verdad bajo contención, incluidos los casos en los que un motor deja de responder, y declara las limitaciones del entorno virtualizado en el que se ejecutó.

Contents

1 Especificaciones del host y del entorno Docker **3**
1.1 Host y máquina virtual . . . 3
1.2 Motores, fijados por digest . . . 3
1.3 Perfiles de recursos . . . 3
1.4 Limitaciones del entorno, declaradas . . . 4
1.5 Consecuencia para la limpieza de caché: permiso de *root* . . . 4

2 Método de medición y verificación del baseline **4**
2.1 El conjunto de datos es el mismo para los dos motores . . . 4
2.2 Las consultas se leen del disco, no están dentro del código . . . 5
2.3 Qué es una repetición . . . 5
2.4 Cómo se resumen las repeticiones (pregunta 5 del revisor) . . . 5
2.5 Cómo se calculan los percentiles (pregunta 2 del revisor) . . . 5
2.6 De dónde sale cada columna del CSV . . . 6
2.7 Detección de suspensión del equipo . . . 6

3 Resultados consolidados **6**
3.1 Qué se consiguió medir . . . 6
3.2 Tabla consolidada . . . 7
3.3 Throttling de CPU (pregunta 3 del revisor) . . . 8
3.4 Estabilidad de las mediciones . . . 8

4 Análisis gráfico **8**

5 Planes de ejecución **10**
5.1 Dos avisos para leerlos . . . 10
5.2 La asimetría de planes, declarada . . . 10
5.3 Lo que el plan explica del cuello de botella (pregunta 6) . . . 12

6 Interpretación **12**
6.1 M1: ¿cómo se degrada cada motor al apretar los recursos? . . . 12
6.2 M2: ¿hubo volcado a disco y OOM, y dónde? . . . 12
6.3 La premisa del manual sobre las dos variantes de `Q2` . . . 13
6.4 M3: huella de CPU y memoria por cada 1000 documentos . . . 13
6.5 Y la conclusión que el throttling deja clara . . . 14

7 Recomendaciones técnicas para despliegues documentales en la nube **14**

A Anexo: las dos variantes de estrés de memoria de `Q2` **15**
A.1 Por qué este anexo existe, y por qué es parcial . . . 15
A.2 Resultado: las dos variantes no se diferencian . . . 15
A.3 Conclusión . . . 15

B Anexo: integridad experimental **16**
B.1 Repeticiones descartadas . . . 16
B.2 Lo que quedó sin explicar . . . 16
B.3 Lo que el entorno no permite afirmar . . . 16
B.4 Lo que sí se verificó antes de medir . . . 16
B.5 Reproducibilidad . . . 16

1 Especificaciones del host y del entorno Docker

1.1 Host y máquina virtual

Las mediciones se ejecutaron en un portátil con Windows 11, WSL2 y Docker Desktop. El repositorio y los volúmenes viven en el sistema de archivos de WSL2, nunca bajo `/mnt/c/`, para no medir la capa de traducción de Windows.

Table 1: Host y máquina virtual. Capturado por `scripts/capture_env.py` en cada repetición; evidencia en `results/env/`.

Procesador	AMD Ryzen 5 PRO 5675U: 6 núcleos físicos, 12 hilos
Memoria del host	14.8 GiB
Sistema	Windows 11 + WSL2 (distribución `Ubuntu`) + Docker Desktop
VM de WSL2	12 CPUs lógicas y 7.159 GiB visibles
cgroups	v2, confirmado leyendo el cgroup desde dentro del contenedor
Reparto de CPUs	motor en las lógicas 0–7; runner y telemetría en 8–11, sin compartir núcleo físico (decisión F5-c, topología medida)

1.2 Motores, fijados por digest

Las imágenes no se referencian por etiqueta sino por *digest*: una etiqueta como `mongo:8.2` es móvil y un `docker pull` posterior podría traer otra versión, lo que rompería la reproducibilidad (R-33).

Table 2: Motores y telemetría. Digests completos en `docker/images.env` y en `results/env/f1/`.

Componente	Versión	Digest (abreviado)
MongoDB	8.2.12 (wiredTiger)	`sha256:e0ce8c35...4bd3`
PostgreSQL	18.6	`sha256:3a426452...4b81`
Prometheus	v3.15.1	`sha256:fa330227...91a2`
cAdvisor	v0.60.6	`sha256:b8e7d109...d07a`
node_exporter	v1.12.1	`sha256:1b4e4438...1be0`

1.3 Perfiles de recursos

Los tres perfiles del manual se aplican con *cgroups* v2 y se **verifican leyendo el cgroup** antes de cada repetición, no se dan por puestos (`scripts/verify_limits.sh`).

Table 3: Perfiles y límites reales observados en el cgroup. Los seis (3 perfiles × 2 motores) se verificaron en F1.

Perfil	CPU	Memoria	`memory.swap.max`
`PROFILE_A_BASELINE`	4.0	4096 m	0
`PROFILE_B_MEDIUM`	2.0	2048 m	0
`PROFILE_C_SEVERE`	1.0	1024 m	0

`memory.swap.max = 0` en los seis casos: cuando un motor agota su límite de memoria no tiene *swap* donde escapar, y el kernel lo mata. Esa es la garantía que sustituye a `mem_swappiness`, que **no existe en cgroups v2**: el manual lo pide y se declara en el *compose*, pero Docker lo descarta con un aviso, así que no es un fallo de la configuración (SPEC A7, V8).

1.4 Limitaciones del entorno, declaradas

El manual supone un host dedicado. Este no lo es, y conviene decirlo antes de los resultados y no después:

1. **La limpieza de caché no llega hasta el disco.** `drop_caches` vacía la caché de páginas de la VM de WSL2, pero **no la de Windows sobre el `.vhdx`** donde viven los volúmenes. Lo alcanzable es un estado de partida *repetible*, no frío. Se midió el efecto en cada repetición (entre 0.7 y 1.8 GB liberados), no solo el código de salida del comando.

2. **El perfil A no es un baseline holgado.** La VM tiene 7.159 GiB en total; con 4 GiB para el motor quedan unos 3 GiB para el runner, la telemetría y la caché de páginas. Se midió: en el peor caso quedaron 3.2 GiB libres, sin *swap*, pero con 120 de 300 s en espera por memoria (F7.4).

3. **Las 4 CPUs del perfil A no son 4 núcleos físicos.** La VM expone 12 lógicas sobre 6 físicos; el reparto 0–7 para el motor usa 4 núcleos físicos completos, medidos con `thread_siblings_list`.

4. **La frecuencia del procesador baja bajo carga sostenida.** Medido durante la matriz: el rendimiento del procesador osciló entre el 86 % y el **60.8 %** de la frecuencia nominal (de 2296 a 1396 MHz). No se puede leer desde WSL2 —no hay `cpufreq`— y se midió desde Windows con el contador «% de rendimiento del procesador». Es la causa probable de la mayor de las dispersiones observadas (sección 3).

5. **cAdvisor observa la VM, no el equipo.** Las curvas de telemetría describen el contenedor dentro de WSL2.

1.5 Consecuencia para la limpieza de caché: permiso de *root*

El flush por repetición necesita *root*. Para que una corrida de horas no se detuviera a pedir contraseña se instaló una regla de `sudo` acotada a **un único ejecutable** (`/usr/local/sbin/bench-drop-caches`), propiedad de `root`, que no acepta parámetros y solo hace `sync` y `drop_caches`. Vive fuera del repositorio a propósito: si estuviera dentro, donde el usuario puede editarlo, la regla equivaldría a *root* sin contraseña. El resto de `sudo` sigue pidiéndola, y la regla se deshace con un script. Cada repetición registra por qué vía se limpió la caché (`"via"` en el archivo crudo). Diseño y reversión en `docs/V7_SUDOERS.md`.

2 Método de medición y verificación del baseline

2.1 El conjunto de datos es el mismo para los dos motores

Los 500000 documentos se generan con `generator/synthetic_data_generator.py` con **semilla fija 42**, de modo que el archivo es reproducible byte a byte. Su huella SHA-256 empieza por `0cd7fadb` y termina en `0414`, y queda registrada en `results/dataset_stats.json` junto con las distribuciones obtenidas.

Que los dos motores contengan lo mismo no se supone: se verificó en tres direcciones (archivo → motor, motor → archivo y motor → motor) con `scripts/verify_gate.py`, comparando los documentos exportados de cada motor contra la fuente en *streaming*. Dos decisiones del esquema son las que hacen que la comparación sea válida:

- `payment.final_total` se almacena como **número** JSON, no como cadena. Si fuera cadena, MongoDB ordenaría lexicográficamente (`"9.5" > "100.0"`) y PostgreSQL numéricamente: los Top-1000 no coincidirían y la comparación quedaría invalidada *en silencio*.

- `timestamp` se almacena como cadena ISO-8601 en los dos motores. Como `Date` en MongoDB, el `$gte` contra un literal de texto no haría *match*.

2.2 Las consultas se leen del disco, no están dentro del código

Las cuatro consultas (`Q1` y `Q2`, para cada motor) se copiaron **literalmente** del manual a `queries/` y el runner las lee de ahí en cada ejecución, con una comprobación de integridad al arrancar. El revisor puede compararlas con el PDF sin leer código.

Su equivalencia se verificó antes de medir: `scripts/check_equivalence.py` calcula el resultado de referencia en Python con aritmética exacta (`Decimal`) y lo compara con lo que devuelve cada motor. Coinciden en todas las variantes. El orden *dentro* del Top-1000 de `Q2` no es determinista —hay valores repetidos y la consulta no tiene desempate—, así que se compara como conjunto.

2.3 Qué es una repetición

1. Se baja el motor que estuviera arriba. **Solo un motor corre a la vez**: con los dos arriba competirían por la caché de páginas de la VM y la comparación dejaría de ser válida.
2. Se limpia la caché y **se mide el efecto** (cuánto bajó `buff/cache`), no solo el código de salida del comando. Entre 0.7 y 1.8 GB liberados por repetición, registrados en el archivo crudo.
3. Se levanta el motor con el perfil de la celda y se espera su *healthcheck*.
4. **Se verifican los límites leyendo el cgroup** y se comparan con los esperados del perfil. Si no coinciden, la repetición se aborta sin escribir fila.
5. Ventanas de **60 s de calentamiento, 300 s de medición y 60 s de enfriamiento**, como fija el manual. Los clientes trabajan en bucle cerrado, sin pausa entre consultas.

La concurrencia 50 se reparte en **10 procesos de 5 hilos**, cada hilo con su propia conexión. Se verificó que el motor ve los 50 clientes simultáneos igual que con 50 procesos, y se eligió este reparto porque con 50 procesos el runner ocupaba unos 2 GB y empujaba la VM a *swap*, frente a unos 460 MB con 10 procesos. La decisión se apoya en el P50, el throughput, la concurrencia vista por el motor, las conexiones y la memoria; **no** en el P95, que quedó contaminado por el arranque en frío.

2.4 Cómo se resumen las repeticiones (pregunta 5 del revisor)

Tres repeticiones por celda, y las tablas de este reporte dan la **media** de las tres. Se eligió la media y no la mediana porque con tres valores la mediana es simplemente el del medio y descarta los otros dos. Para que el promedio no esconda dispersión, cada celda se acompaña de su coeficiente de variación, y las celdas cuya dispersión supera el 15 % o que muestran una tendencia monótona se señalan una por una en la sección 3.

2.5 Cómo se calculan los percentiles (pregunta 2 del revisor)

Se reportan **P50, P95 y P99**, no promedios de latencia. Se calculan con interpolación lineal (`statistics.quantiles` con `method="inclusive"`, el mismo método que `numpy` usa por defecto) sobre las latencias de las consultas **completadas dentro de la ventana de medición**. Las latencias crudas de cada consulta se conservan en `results/raw/`, así que los percentiles se pueden recalcular con otro método sin volver a medir.

Los *timeouts* son datos censurados, no ceros. Una consulta que agota el límite de 300 s no se descarta ni se cuenta como un valor: es un dato **censurado por la derecha**, del que solo se sabe que su latencia es mayor que el límite. Por eso, cuando la fracción de *timeouts* dentro de la ventana alcanza el 1 %, la fila se marca con `status = error` y **cada percentil se declara válido o no**: el P99 solo vale si los *timeouts* son menos del 1 %, el P95 menos del 5 % y el P50 menos del 50 %. Esa validez va fila por fila en `results/raw/`. En este experimento 4 celdas quedaron con percentiles invalidados, y se marcan en todas las tablas.

Celdas con pocas muestras. Con 300 s de ventana y consultas de 20 s, una celda puede completar 15 consultas. Se marcan como **baja densidad muestral** (menos de 20 completadas) y su P99 se lee con esa reserva, en vez de presentarlo como si viniera de cientos de muestras.

2.6 De dónde sale cada columna del CSV

Las 14 columnas se miden directamente, y **ninguna se rellena con un cero inventado**: si una lectura falta, la celda queda vacía y la fila sale con `status error` u `oom`.

Table 4: Origen de las columnas que no son identificadores.

`throughput_qps`	consultas completadas en la ventana ÷ 300 s
`latency_p50/p95/p99_ms`	percentiles de las completadas en la ventana
`cpu_throttled_seconds`	Δ `throttled_usec` de `cpu.stat` del cgroup ÷ 1e6
`max_rss_mb`	máximo de `anon` de `memory.stat`, muestreado cada segundo durante la ventana
`disk_spill`	Δ `temp_bytes` (PostgreSQL) o contadores de *spill* de `serverStatus` (MongoDB)
`status`	`oom` > `error` > `spill` > `ok`

El cgroup se lee **desde fuera del contenedor**, en `/sys/fs/cgroup/docker/`, y no con `docker exec`: se midió que cada lectura por `exec` cuesta unos 250 ms y gasta CPU *del motor*, y muestrear así cada segundo en el perfil C se comería un 8 % de su cuota. Es decir, mediríamos la medición.

Jerarquía de fuentes. Las columnas del CSV salen del cgroup y de los motores. Prometheus, cAdvisor y node_exporter (con `scrape_interval: 1s`) alimentan las curvas y sirven de control cruzado, pero **no** producen ninguna columna. Se verificó que las dos fuentes miden lo mismo: `container_memory_rss` de cAdvisor es exactamente `anon` del cgroup —comprobado con el contenedor congelado, donde ningún proceso puede alterar la memoria— y el throttling de las dos fuentes difiere menos del 1 % bajo carga.

2.7 Detección de suspensión del equipo

Si el equipo se suspende durante una ventana, la VM se congela: las consultas en vuelo se detienen y el reloj de pared avanza sin que corra nada. Esa ventana ya no mide lo que dice medir. El runner lo detecta comparando el reloj de pared con el monótono durante la ventana y marca la repetición como `error`. En esta matriz **no ocurrió ninguna vez**, y el número de repeticiones descartadas por cualquier motivo es 0 (sección B).

3 Resultados consolidados

3.1 Qué se consiguió medir

La matriz son 36 celdas (2 motores × 3 perfiles × 2 consultas × 3 concurrencias) con 3 repeticiones cada una: 108 filas, todas en `results/measurements.csv`, validadas contra el esquema

exacto de 14 columnas del manual. Ninguna combinación falta ni está repetida, y se descartaron 0 repeticiones.

Table 5: Reparto de las 108 filas por `status`. La precedencia es `oom` > `error` > `spill` > `ok`, así que una fila `oom` puede tener además volcado a disco: por eso 24 filas tienen `disk_spill = true` y solo 18 salen con `status = spill`.

`status`	Filas	Qué significa
`ok`	78	la celda se midió sin incidencias
`spill`	18	hubo volcado a disco durante la ventana
`error`	7	percentiles invalidados por censura (A4)
`oom`	5	el kernel mató algún proceso del motor

De las 36 celdas, 32 son **medibles** (sus tres repeticiones salieron `ok` o `spill`). Las otras cuatro contienen filas `error` u `oom`: en ellas el motor no completó trabajo suficiente como para que un percentil signifique algo, y **eso es el resultado**, no un fallo del instrumento.

3.2 Tabla consolidada

Table 6: Resultados por celda: media de las 3 repeticiones. Latencias en ms, throttling en segundos por ventana de 300 s, `max_rss_mb` en MiB.

Motor	Perfil	Consulta	c	qps	P50	P95	P99	Throttl.	RSS	Spill	`status`
MongoDB	A	Q1	1	0.2911	3414	3743	4011	0.0	823.6	no	ok
MongoDB	A	Q1	10	0.5444	18149	21824	23459	1180.8	872.9	no	ok
MongoDB	A	Q1	50	0.6856	74756	83773	85571	1185.2	1040.0	no	ok
MongoDB	A	Q2	1	0.7300	1352	1526	1704	0.0	831.9	no	ok
MongoDB	A	Q2	10	1.9078	5110	6979	7779	1186.5	861.2	no	ok
MongoDB	A	Q2	50	2.2711	21775	24751	26324	1196.1	961.9	no	ok
MongoDB	B	Q1	1	0.2611	3818	4019	4167	12.8	812.2	no	ok
MongoDB	B	Q1	10	0.3300	30597	35030	37341	1783.6	822.0	no	ok
MongoDB	B	Q1	50	0.3333	138931	148438	151839	1788.0	879.4	no	ok
MongoDB	B	Q2	1	0.1800	5536	5934	6130	17.4	829.8	no	ok
MongoDB	B	Q2	10	0.4111	23692	26105	26166	1502.9	805.5	no	ok
MongoDB	B	Q2	50	0.3733	146673	172727	179510	1773.4	908.8	no	ok
MongoDB	C	Q1	1	0.1933	5505	5956	6082	158.2	587.7	no	ok
MongoDB	C	Q1	10	0.1800	57199	63125	64473	2081.3	607.3	no	ok
MongoDB	C	Q1	50	0.1356	283597	296988	298965	2085.2	634.7	no	error‡
MongoDB	C	Q2	1	0.0533	19329	20461	20683	183.4	611.6	no	ok*
MongoDB	C	Q2	10	0.1000	96905	113894	113895	1830.0	561.6	no	ok
MongoDB	C	Q2	50	0.1667	225253	225372	228109	1976.7	672.3	no	ok
PostgreSQL	A	Q1	1	0.1322	7481	8321	9030	0.3	103.8	sí	spill
PostgreSQL	A	Q1	10	0.1444	74687	97188	98180	826.9	570.9	sí	spill
PostgreSQL	A	Q1	50	0.0022	275776	283670	284371	552.6	1851.6	sí	error‡
PostgreSQL	A	Q2	1	1.8800	513	715	795	18.4	15.6	no	ok
PostgreSQL	A	Q2	10	2.3511	3507	7477	8168	907.7	43.4	no	ok
PostgreSQL	A	Q2	50	1.5789	35086	43086	50592	1093.6	127.3	no	ok
PostgreSQL	B	Q1	1	0.1100	8923	10147	11198	146.0	105.0	sí	spill
PostgreSQL	B	Q1	10	0.1000	114076	118444	119141	1535.2	571.0	sí	spill
PostgreSQL	B	Q1	50	0.0133	208046	262901	270597	1216.0	1800.0	sí	error/o

Motor	Perfil	Consulta	c	qps	P50	P95	P99	Throttl.	RSS	Spill	status
PostgreSQL	B	Q2	1	1.0889	885	1130	1311	550.0	16.3	no	ok
PostgreSQL	B	Q2	10	1.1478	7330	15423	17939	1379.2	43.4	no	ok
PostgreSQL	B	Q2	50	1.0556	51582	59091	62590	1634.3	127.3	no	ok
PostgreSQL	C	Q1	1	0.0567	17555	18464	18777	489.3	104.4	sí	spill*
PostgreSQL	C	Q1	10	0.0333	263918	267051	267746	1829.8	564.7	sí	spill*
PostgreSQL	C	Q1	50	0.0000	—	—	—	105.5	475.1	no	oom†
PostgreSQL	C	Q2	1	0.5722	1730	2024	2108	870.7	16.7	no	ok
PostgreSQL	C	Q2	10	0.6000	11936	28847	29888	1622.2	43.8	no	ok
PostgreSQL	C	Q2	50	0.5611	100284	124632	134570	1890.9	127.0	no	ok

† El motor estaba **colapsado** durante la ventana: sus errores son conexiones rechazadas, no consultas ejecutadas, y su `max_rss_mb` **no es eficiencia** (R-29.2).

‡ Percentiles invalidados por censura por la derecha: la fracción de *timeouts* supera el límite de A4.

* Baja densidad muestral: menos de 20 consultas completadas en la ventana (A4).

3.3 Throttling de CPU (pregunta 3 del revisor)

Sí hubo throttling, y es la métrica que más fácil se malinterpreta. Dos advertencias necesarias para leer la columna:

1. **Puede superar la duración de la ventana.** El kernel suma el tiempo de throttling de **cada CPU** en la que el cgroup tenía hilos listos para correr. Con 4 CPUs y una cuota de 1, cada segundo de reloj puede acumular hasta 4 segundos de throttling. El máximo observado es de **2086 s en una ventana de 300 s**. No es un error de medida: cAdvisor reporta la misma suma, y las dos fuentes coinciden al 1 %.
2. **Una parte no la causa el benchmark.** Se midió el throttling de cada motor **en reposo**, sin consultas, por perfil: MongoDB acumula unos 27 s por ventana de 300 s en el perfil C y 3.6 s en el B, mientras PostgreSQL se queda por debajo de 0.5 s en todos. Esa línea base está en `results/f6/f6_3/baseline/` y aparece en la figura 2.

3.4 Estabilidad de las mediciones

Antes de gastar las horas de la matriz se corrió una celda piloto completa y se exigió que el coeficiente de variación del throughput entre las tres repeticiones fuera menor del 15 %; salió del 4.2 % en MongoDB y del 3.8 % en PostgreSQL. Sobre la matriz entera, **la dispersión se mantuvo baja en casi todas las celdas**, pero no en todas, y conviene decir cuáles:

- **Dos celdas superan el 15 %**, las dos de `Q2` con c=50: PostgreSQL en el perfil A (20.9 %, con una caída del 34 % entre la primera y la tercera repetición) y MongoDB en el perfil B (24.0 %, con una caída del 33 %).
- En la de **PostgreSQL** la causa es medible: las tres repeticiones gastaron *el mismo* tiempo de CPU (1197 s) y el rendimiento del procesador cayó del 75.9 % al 65.2 % justo en la repetición que pierde el 34 %. El mismo CPU, más lento, hace menos trabajo.
- En la de **MongoDB** la causa **no está identificada**: el tiempo de CPU es idéntico (598.6 s) y el reloj está plano (83.4 %, 81.7 %, 84.6 %). La caché de Windows sobre el disco virtual, que `drop_caches` no alcanza, es el sospechoso natural, pero no hay medición que lo demuestre y no se presenta como explicación.
- Se comprobó que **no hay un sesgo sistemático** de «la primera repetición es más rápida»: ocurre en 6 de las 30 celdas con las tres repeticiones comparables, cuando por azar serían unas 10.

La consecuencia práctica es que la celda de `Q2` con c=50 en el perfil A es la **única** en la que MongoDB supera a PostgreSQL en `Q2`, y es también una de las dos celdas con dispersión alta. Esa comparación concreta no se usa para sostener ninguna conclusión (sección 6).

4 Análisis gráfico

Las tres figuras las dibuja LaTeX leyendo los archivos de `report/datos/`, que genera `scripts/build_report_data.py` del CSV y de `results/raw/`. Así cada punto es rastreable hasta su fila.

La figura 1 lleva **dos series por panel** y no una nota al pie, porque la diferencia entre ellas no es un detalle: el calentamiento de 60 s que fija el manual no alcanza cuando 50 clientes empiezan a la vez con la caché recién

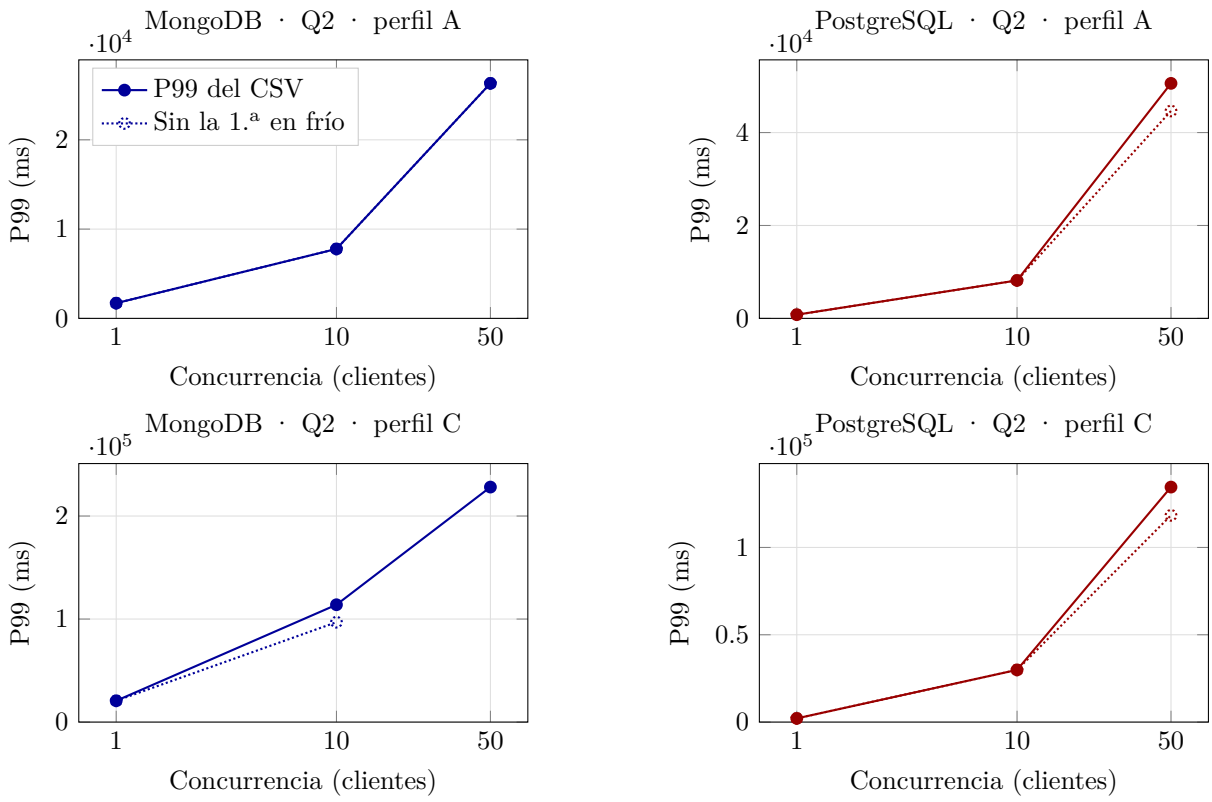


Figure 1: **P99 contra concurrencia, en dos vistas (R-30.1).** La línea continua es el P99 tal como lo mide el manual (columna del CSV). La punteada es el mismo P99 **sin la primera consulta de cada cliente**, recalculado desde `results/raw/`. La separación entre las dos líneas es el arranque en frío: el calentamiento de 60 s del manual no alcanza con c=50 en los perfiles B y C, donde hasta el 91 % de la ventana son primeras consultas (F7.3). En MongoDB · perfil C · c=50 la línea punteada se interrumpe porque *todas* las consultas de la ventana eran primeras consultas y no existe un P99 en régimen que dibujar. **Leer solo la línea continua sobreestima la latencia del motor en esos puntos.** Datos: `report/datos/p99_*.dat`.

limpiada. Se midió una por una la primera consulta de cada cliente y, con c=50, **no cabe en los 60 s en 14 de las 18 combinaciones**; solo cabe en el perfil A con `Q2` (de 39 a 52 s). En los perfiles B y C llega a 110 s, y en algunas celdas ninguna consulta terminó siquiera en los 110 s que dura toda la corrida de prueba.

El caso extremo aparece en la figura: en **MongoDB, perfil C, `Q2`, c=50** las 150 consultas que terminaron dentro de la ventana son *todas* la primera de su cliente, así que no existe un P99 «en régimen» que dibujar y la serie punteada se interrumpe. Decir que el P99 de esa celda es de 228 s sin esta advertencia sería atribuirle al motor el costo de arrancar en frío.

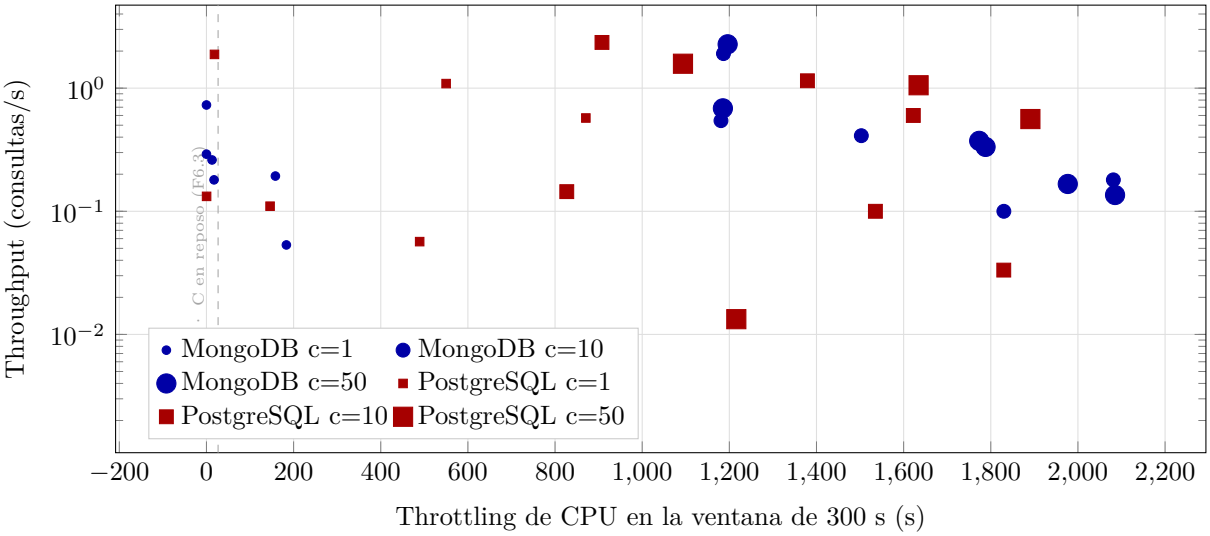


Figure 2: **Throttling de CPU contra throughput.** Cada punto es una celda (media de sus 3 repeticiones); el tamaño de la marca crece con la concurrencia. **El eje horizontal supera los 300 s de la ventana** porque `cpu_throttled_seconds` suma el tiempo de cada CPU en la que el cgroup tenía hilos listos (medido en F6.2, declarado en SPEC §4); el máximo observado es de 2086 s. La línea vertical marca el throttling que MongoDB acumula **en reposo** en el perfil C, sin benchmark (F6.3): a su izquierda no hay contención atribuible a la carga. Datos: `report/datos/throttling_*.dat`.

La figura 3 es la que obliga a tener cuidado con la columna `max_rss_mb`, y por eso lleva dos series: véase la sección 6.4.

5 Planes de ejecución

Se capturó el plan de cada combinación de motor, consulta y perfil con `explain` en modo `executionStats` (MongoDB) y `EXPLAIN (ANALYZE, BUFFERS)` (PostgreSQL). Los 12 planes crudos están en `results/explains/`.

5.1 Dos avisos para leerlos

Los planes se capturan con un solo cliente. Se toman en las celdas de c=1 y **fuera** de la ventana de medición, para no alterar lo que se mide. Eso tiene una consecuencia que hay que declarar: **el plan de c=1 no es el que se ejecuta con c=50**. Con 50 clientes simultáneos, PostgreSQL agota los procesos de trabajo paralelos disponibles y los clientes que llegan después ejecutan la consulta sin paralelismo. El plan documenta la *forma* de resolver la consulta, no el grado de paralelismo que tuvo cada cliente bajo carga.

En `EXPLAIN (BUFFERS)` los contadores incluyen a los hijos. El total es el del nodo raíz. Sumar los nodos cuenta lo mismo varias veces: al verificar las consultas se registraron 8310 bloques donde en realidad eran 1662.

5.2 La asimetría de planes, declarada

`Q2` filtra por `timestamp` y ordena por `final_total`. Los dos motores la resuelven de forma distinta:

- **MongoDB** usa su índice sobre `timestamp` para el `$match`.

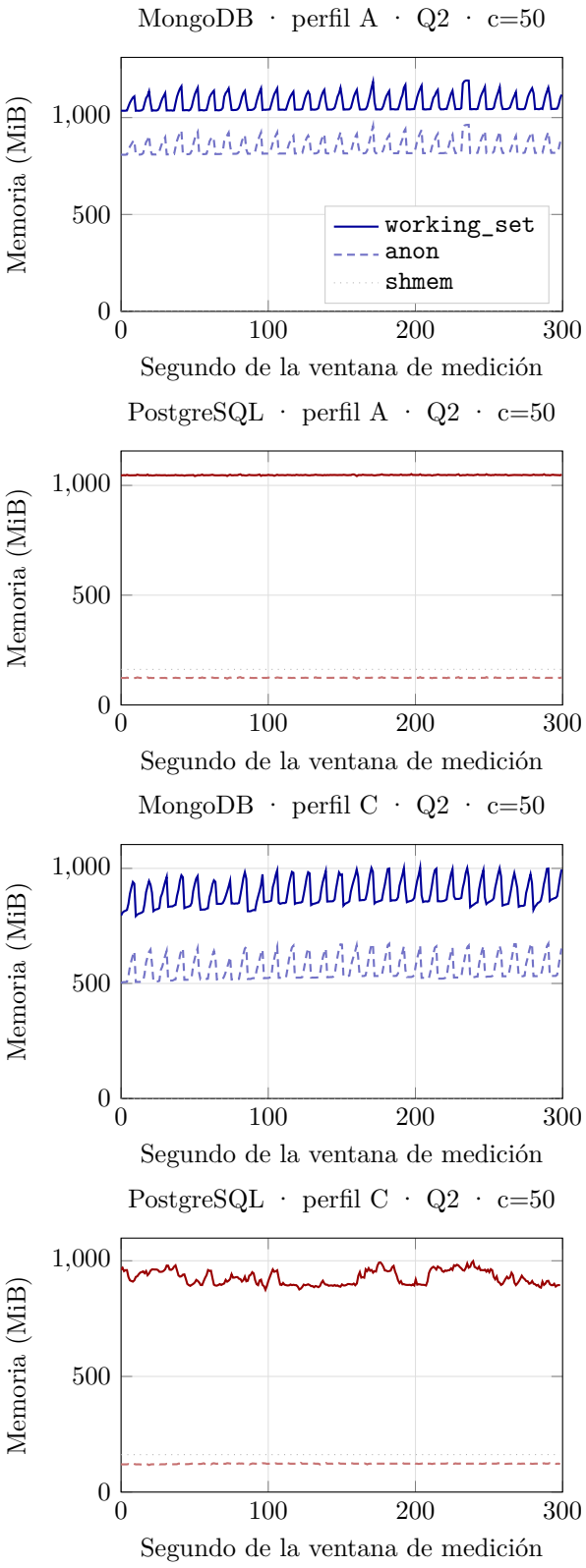


Figure 3: **Memoria del contenedor durante la ventana, muestreada cada segundo.** La línea discontinua (`anon`) es exactamente lo que resume la columna `max_rss_mb`; la continua (`working_set`) incluye además la memoria compartida. **En PostgreSQL la distancia entre las dos es `shared_buffers`**: son `shmem`, no `anon`, así que `max_rss_mb` no cuenta su búfer principal y lo hace parecer artificialmente eficiente frente a MongoDB, cuyo caché de WiredTiger sí es `anon` (F6-A3, SPEC §4 y A11). Por eso M3 se interpreta con `working_set`. Fuente: serie de 1 s del cgroup en `results/raw/`, las mismas lecturas de las que sale la columna.

- **PostgreSQL** hace `Seq Scan`. El índice de expresión existe —se creó sobre el `timestamp` extraído como texto, sin modificar la consulta del manual— pero el planificador no lo usa, porque la mitad de la tabla pasa el filtro y recorrerla entera le sale más barato que saltar por el índice.

Esto es importante al interpretar las latencias: **en parte se están comparando dos planes distintos**, no solo dos motores. No se «arregló» forzando el índice, porque eso significaría medir una consulta que no es la del manual.

5.3 Lo que el plan explica del cuello de botella (pregunta 6)

`Q1` en PostgreSQL: el plan explica el volcado. El plan muestra el ordenamiento con `Sort Method: external merge` y `Sort Space Type: Disk`, que es exactamente lo que registran los contadores `temp_bytes` de la sección 6.2. Las dos fuentes coinciden: no es una conjetura sobre por qué `Q1` es lenta en PostgreSQL.

`Q2` en los dos motores: el plan explica por qué *no* hay volcado. El `LIMIT 1000` convierte el ordenamiento en un *top-K*: no hace falta ordenar las 175049 filas que pasan el filtro, solo mantener las 1000 mejores. PostgreSQL lo resuelve con `top-N heapsort` y `Sort Space Used: 273` kB **en memoria**. Esa cifra es la clave de la sección 6.3: 273 kB caben de sobra tanto en los 4 MB de `work_mem` de la variante restrictiva como en los 64 MB de la permisiva, y por eso las dos variantes de `Q2` no se diferencian.

6 Interpretación

6.1 M1: ¿cómo se degrada cada motor al apretar los recursos?

El resultado no se parte por motor, se parte **por consulta**, y es consistente en los tres perfiles y en las tres concurrencias.

Table 7: Quién gana cada celda, en throughput (media de 3 repeticiones). El factor es cuántas veces más consultas por segundo completa el ganador.

Consulta	Perfil	c=1	c=10	c=50
`Q1`	A	Mongo ×2.2	Mongo ×3.8	PostgreSQL no completa[‡]
`Q1`	B	Mongo ×2.4	Mongo ×3.3	Mongo ×25
`Q1`	C	Mongo ×3.4	Mongo ×5.4	los dos fallan[†]
`Q2`	A	PostgreSQL ×2.6	PostgreSQL ×1.2	Mongo ×1.4[*]
`Q2`	B	PostgreSQL ×6.0	PostgreSQL ×2.8	PostgreSQL ×2.8
`Q2`	C	PostgreSQL ×10.7	PostgreSQL ×6.0	PostgreSQL ×3.4

`Q1` (desanidar y agrupar): MongoDB gana siempre. Y la ventaja **crece** al apretar los recursos: de ×2.2 en el perfil A a ×3.4 en el C con un cliente. La razón está en el plan y en el disco: para PostgreSQL, `Q1` exige un ordenamiento que no cabe en `work_mem` y termina en disco **en los tres perfiles** (sección 6.2), mientras MongoDB la resuelve sin tocar disco.

`Q2` (ordenar y quedarse con el Top-1000): PostgreSQL gana, y la ventaja crece al apretar. De ×2.6 a ×10.7 con un cliente al pasar del perfil A al C. Visto al revés: entre el perfil A y el C, MongoDB se vuelve **13.7 veces** más lento en `Q2` y solo 1.5 veces más lento en `Q1`. La sensibilidad a la contención depende más de la consulta que del motor.

La única excepción, y por qué no se usa. La celda `Q2` · perfil A · c=50 es la única de `Q2` que gana MongoDB, y es también **una de las dos celdas con dispersión por encima del 15 %** (sección 3.4): su medición de PostgreSQL cayó un 34 % entre la primera y la tercera repetición mientras el procesador bajaba de frecuencia. Esa inversión no se usa para sostener nada.

6.2 M2: ¿hubo volcado a disco y OOM, y dónde?

Volcado: sí, masivo, y solo en PostgreSQL con `Q1`. 24 de las 108 filas tienen `disk_spill = true`, y **todas son de PostgreSQL con `Q1`**: 8 de sus 9 celdas (la novena no volcó porque el motor murió antes). En total, **0.58 TB** de archivos temporales escritos a lo largo de la matriz. Con un solo cliente en el perfil A son unos 32 GB por ventana de 300 s, es decir **unos 0.8 GB por consulta**.

MongoDB no volcó en ninguna celda, ni siquiera con `allowDiskUse: true` (anexo 6.3).

OOM: cinco filas, y hay que distinguir dónde ocurrió cada uno. Son 5 filas con `status = oom`, todas de PostgreSQL con `Q1` y c=50 en los perfiles B y C. Presentarlas como «cinco OOM» sin más sería impreciso:

- **2 ocurrieron dentro de la ventana de medición**: el contador `oom_kill` del cgroup se movió mientras se medía.
- **3 ocurrieron durante el calentamiento**: el contador ya estaba en su valor final al abrirse la ventana. El motor llegó a la medición *ya* sin memoria.

La distinción importa porque cambia lo que la fila mide. Las tres de la segunda clase no describen «un motor que se queda sin memoria atendiendo la carga medida», sino «un motor que ya se había quedado sin memoria antes de empezar a medir». El detalle de cada una está en `results/logs/`.

Table 8: Las 5 repeticiones con `status = oom`. La columna «Δ `oom_kill`» es cuántos procesos mató el kernel *durante la ventana*: un cero con `OOMKilled = sí` significa que el OOM ocurrió antes, en el calentamiento (R-29.1).

Motor	Perfil	Consulta	c	Rep.	Δ `oom_kill`	Cuándo	Completadas	`OOMKilled`
PostgreSQL	B	Q1	50	1	0	en el calentamiento	4	sí
PostgreSQL	B	Q1	50	2	0	en el calentamiento	4	sí
PostgreSQL	C	Q1	50	1	0	en el calentamiento	0	sí
PostgreSQL	C	Q1	50	2	1	en la ventana	0	sí
PostgreSQL	C	Q1	50	3	6	en la ventana	0	sí

La peor celda, y cómo no leerla. En **PostgreSQL · perfil C · `Q1` · c=50** no se completó *ninguna* consulta en las tres repeticiones: el throughput es 0.000. La fila registra `cpu_throttled_seconds = 0.035` y `max_rss_mb = 8.2`[†], y esos dos números **no significan que PostgreSQL atendiera 50 clientes con 8 MiB de memoria**: significan que durante la ventana el motor estaba colapsado y no trabajaba. Sus 15000 errores registrados son **conexiones rechazadas**, no consultas ejecutadas. Por eso esa celda lleva nota al pie en todas las tablas de este reporte.

Respuesta a M2. El volcado a disco es real, sistemático y enorme, pero está concentrado en una sola combinación: PostgreSQL con `Q1`. El OOM aparece solo donde se juntan la consulta más pesada, la concurrencia más alta y los perfiles con menos memoria. MongoDB no volcó ni fue matado por el kernel en ninguna celda.

6.3 La premisa del manual sobre las dos variantes de `Q2`

El manual pide dos variantes de estrés de memoria para `Q2` (`work_mem` de 4 MB frente a 64 MB en PostgreSQL; `allowDiskUse` falso frente a verdadero en MongoDB) **dando por supuesto que se diferenciarían**. Se midieron las dos en el perfil con menos memoria y **no se diferencian**: ni en throughput más allá de la dispersión propia de cada celda, ni en volcado, que no aparece en ninguna de las dos.

La explicación está en el plan (sección 5): el `LIMIT 1000` convierte el ordenamiento en un *top-K* de **273 kB**, que caben de sobra en los 4 MB de la variante restrictiva. El parámetro que el manual varía nunca llega a ser el límite. El detalle de la comparación está en el anexo A, con la cobertura parcial declarada.

6.4 M3: huella de CPU y memoria por cada 1000 documentos

El CSV no tiene un denominador, así que se toma de los **planes de ejecución**, como corresponde: los documentos que cada consulta examina de verdad. Y ahí aparece una asimetría que hay que declarar antes de dividir:

Motor	Consulta	Documentos examinados por consulta
MongoDB	`Q1`	500000 (recorrido completo)
MongoDB	`Q2`	250207 (usa su índice de `timestamp`)
PostgreSQL	`Q1`	500000
PostgreSQL	`Q2`	500000 (`Seq Scan`: no usa el índice)

CPU por 1000 documentos (perfil A, un cliente). MongoDB gasta 8.6 ms en `Q1` y PostgreSQL 25.6 ms: **MongoDB es unas tres veces más eficiente en CPU** para esa consulta. En `Q2` se invierte: PostgreSQL gasta 3.5 ms y MongoDB 6.6 ms. Y al apretar los recursos la brecha de `Q2` se abre hasta ser de **74.7 ms frente a 3.6 ms** en el perfil C, unas 21 veces.

La memoria depende de cuál de las dos memorias se mire, y esto es importante. `max_rss_mb` mide la memoria *anónima* del contenedor. Los `shared_buffers` de PostgreSQL son memoria **compartida**, que el kernel no cuenta como anónima; el caché de WiredTiger de MongoDB sí lo es. El efecto es grande:

	`max_rss_mb` (anónima)		`working_set`	
Perfil A, c=1	`Q1`	`Q2`	`Q1`	`Q2`
MongoDB	824	832	1042	1058
PostgreSQL	104	16	1630	891

Leyendo solo `max_rss_mb`, PostgreSQL parecería de 8 a 50 veces más eficiente en memoria. Leyendo `working_set` —que sí incluye la memoria compartida y es la que cuenta contra el límite del contenedor— **la relación se invierte en `Q1`**: 1630 MiB de PostgreSQL frente a 1042 de MongoDB. Por eso M3 se responde con `working_set`, y la figura 3 muestra las dos curvas juntas.

Un matiz honesto sobre `working_set`. Incluye también la caché de páginas de los archivos de datos que se le atribuye al contenedor. PostgreSQL depende de ella por diseño —lee a través de la caché del sistema operativo— y cuenta contra su `memory.max`, así que es memoria que el motor necesita bajo el límite; pero no es «memoria del proceso». Las dos lecturas están en los datos (`report/datos/m3_huella.dat`) para que cualquiera pueda rehacer la cuenta con el criterio que prefiera.

6.5 Y la conclusión que el throttling deja clara

El throttling confirma que los perfiles del manual **aprietan de verdad**: hasta 2086 s acumulados en una ventana de 300 s. Pero no es lo que diferencia a los motores. Lo que los diferencia es si la consulta cabe en memoria: donde cabe (`Q2`, por el *top-K*), gana el motor con el plan más eficiente; donde no cabe (`Q1` en PostgreSQL), el disco domina y la CPU pasa a segundo plano.

7 Recomendaciones técnicas para despliegues documentales en la nube

Las recomendaciones salen de lo medido aquí, con sus límites: un solo conjunto de datos de 500000 documentos, dos consultas y un host virtualizado. No son afirmaciones generales sobre los dos motores.

1. Elegir el motor por la forma de la consulta, no por el motor. Es el resultado más claro: MongoDB gana `Q1` en las nueve celdas y PostgreSQL gana `Q2` en ocho de nueve. Si la carga dominante desanida arreglos y agrupa, MongoDB rinde de 2 a 5 veces más; si ordena y se queda con un Top-K, PostgreSQL rinde de 1.2 a 10.7 veces más. Un despliegue que haga las dos cosas no se resuelve eligiendo motor, sino decidiendo qué consulta importa más.

2. Dimensionar `work_mem` mirando el plan, no el tamaño de la tabla. `Q2` ordena 175049 filas y necesita **273 kB**, porque el `LIMIT` la convierte en un *top-K*. `Q1` sobre los mismos datos necesita unos 0.8 GB **por consulta** y vuelca a disco con cualquier valor razonable de `work_mem`. Subir el parámetro de 4 a 64 MB no cambió nada medible en `Q2` y no habría salvado a `Q1`: lo que decide es la forma del plan.

3. En PostgreSQL, vigilar `temp_bytes` como métrica de primera línea. El volcado no se nota en la CPU ni en el RSS: se nota en **0.58 TB** escritos en disco a lo largo del experimento. Una consulta que vuelca degrada por una vía que el throttling de CPU y la memoria residente no muestran, y `pg_stat_database.temp_bytes` la delata de inmediato.

4. No usar la memoria residente para dimensionar un contenedor de PostgreSQL. `max_rss_mb` no cuenta los `shared_buffers`, que son memoria compartida. En este experimento PostgreSQL aparece con 16 MiB residentes y un conjunto de trabajo de 891 MiB en la misma celda. Un límite de memoria fijado con la primera cifra mataría el contenedor de inmediato. La métrica útil es `working_set` (o `memory.current` menos la caché recuperable).

5. Con límites de CPU bajos, el calentamiento tiene que ser largo. Con 50 clientes y la caché limpia, la primera consulta de cada cliente tarda de 69 a 110 s en los perfiles B y C. Un calentamiento de 60 s deja ese costo dentro de la ventana de medición y contamina los percentiles de cola: medido, hasta el 91 % de las consultas de una ventana eran primeras consultas. Quien mida latencia de cola bajo contención debe comprobar que su calentamiento cubre la primera consulta, no suponerlo.

6. Un límite de memoria sin *swap* convierte la saturación en caída, no en lentitud. Con `memory.swap.max = 0`, PostgreSQL con `Q1` y 50 clientes no se degradó: fue **matado por el kernel**, y en tres de las cinco veces antes de empezar a medir. Para un servicio en producción eso es la diferencia entre responder despacio y no responder. Si el objetivo es degradar en vez de caer, hace falta limitar la concurrencia en la capa de conexión antes de que el motor agote su límite.

7. Medir en un portátil obliga a declarar la frecuencia del procesador. Durante la matriz el rendimiento del procesador osciló entre el 86 % y el 60.8 % de la frecuencia nominal, y la mayor dispersión observada coincide con una de esas caídas. En un host dedicado esto no aparecería; en un entorno como este, hay que medirlo y declararlo, no suponer que la CPU es constante.

A Anexo: las dos variantes de estrés de memoria de `Q2`

A.1 Por qué este anexo existe, y por qué es parcial

El manual (§5.2) pide ejecutar `Q2` con dos configuraciones de memoria. El conteo de escenarios que fija el propio manual son 36 celdas, y con 36 el esquema del CSV **no tiene ninguna columna donde distinguir la variante**: añadir una está prohibido explícitamente. Las dos exigencias no caben juntas.

La decisión tomada fue medir la segunda variante **en anexo**, fuera del CSV de las 36 celdas, y solo en `PROFILE_C_SEVERE`: los dos motores y las tres concurrencias, 6 celdas y 18 repeticiones. El razonamiento:

- `PROFILE_C` es el perfil con **menos memoria** (1 GiB). Si las dos variantes no se diferencian ahí, no se diferencian en ninguno.
- Por el plan ya se sabía que el ordenamiento de `Q2` ocupa **273 kB** (sección 5), muy por debajo tanto de los 4 MB de la variante restrictiva como de los 64 MB de la permisiva.

La cobertura es parcial y se declara como tal: no se midió `Q2_MEM_HIGH` en los perfiles A y B. Los datos están en `results/f8_anexo/measurements.csv` (18 filas, validadas con el mismo esquema de 14 columnas).

A.2 Resultado: las dos variantes no se diferencian

Table 9: `Q2` en `PROFILE_C_SEVERE`: variante restrictiva (`work_mem` 4 MB / `allowDiskUse` falso) frente a permisiva (64 MB / verdadero). Throughput medio de 3 repeticiones.

Motor	c	Restrictiva	Permisiva	Diferencia	Volcado
MongoDB	1	0.0533	0.0444	−16.7 %	ninguno en ninguna
MongoDB	10	0.1000	0.1000	0.0 %	ninguno en ninguna
MongoDB	50	0.1667	0.1667	0.0 %	ninguno en ninguna
PostgreSQL	1	0.5722	0.6622	+15.7 %	ninguno en ninguna
PostgreSQL	10	0.6000	0.6222	+3.7 %	ninguno en ninguna
PostgreSQL	50	0.5611	0.5744	+2.4 %	ninguno en ninguna

Las dos diferencias mayores (un −16.7 % en MongoDB y un +15.7 % en PostgreSQL, las dos con un cliente) **apuntan en direcciones opuestas**, que es la forma que tiene el ruido, no un efecto del parámetro. Las otras cuatro están entre el 0 y el 3.7 %, por debajo de la dispersión propia de esas celdas. Las 18 repeticiones salieron `ok` y **ninguna volcó a disco**, ni con `allowDiskUse: true` en MongoDB ni con 4 MB de `work_mem` en PostgreSQL.

A.3 Conclusión

La premisa del manual no se cumple. Se esperaba que la variante restrictiva forzara volcado a disco y que las dos configuraciones dieran resultados distintos; medidas las dos en el perfil más exigente, no se diferencian y ninguna vuelca. La causa está en el plan y no en el parámetro: el `LIMIT 1000` acota el ordenamiento a 273 kB, así que el límite de memoria que el manual varía nunca se alcanza. Se reporta como hallazgo, no como fallo del experimento.

B Anexo: integridad experimental

B.1 Repeticiones descartadas

Se descartaron **0** repeticiones. Ninguna. La herramienta para descartarlas existe (`scripts/descartar_repeticion.py`), exige un motivo escrito y deja la fila descartada registrada en un archivo aparte junto con su crudo, precisamente para que descartar no pueda ser silencioso. No hizo falta usarla: el equipo no se suspendió durante ninguna ventana y ninguna repetición quedó inservible.

B.2 Lo que quedó sin explicar

Una celda con dispersión alta y sin causa identificada. En **MongoDB · perfil B · `Q2` · c=50** el throughput cayó un 33 % entre la primera repetición y las otras dos, con un coeficiente de variación del 24 %. Las tres repeticiones gastaron *el mismo* tiempo de CPU (598.6 s) y la frecuencia del procesador estuvo plana (83.4 %, 81.7 %, 84.6 %). Es decir: el mismo procesador, a la misma velocidad, hizo un 33 % menos de trabajo. La caché de Windows sobre el disco virtual —que `drop_caches` no alcanza— es el sospechoso natural, pero **no hay medición que lo demuestre** y no se presenta como explicación. La celda se reporta con su dispersión a la vista.

La frecuencia del procesador no se puede medir desde dentro. WSL2 no expone `cpufreq` y `/proc/cpuinfo` informa siempre la frecuencia nominal. Se midió desde Windows con el contador «% de rendimiento del procesador», que es un indicador agregado del sistema, no una lectura por núcleo sincronizada con cada ventana. Sirve para detectar que la frecuencia baja; no para atribuir con precisión cuánto de cada medición se debe a ello.

B.3 Lo que el entorno no permite afirmar

Las cinco limitaciones del entorno están en la sección 1.4. La que más afecta a la interpretación: **el estado de partida de cada repetición es repetible, no frío**. La caché de páginas de la VM se limpia y se mide cuánto se liberó, pero la caché de Windows sobre el `.vhdx` no es alcanzable. Un experimento en un host dedicado podría dar números distintos; lo que este experimento garantiza es que **las dos mediciones que se comparan partieron del mismo estado**, no que partieran de disco frío.

B.4 Lo que sí se verificó antes de medir

Para que lo anterior no se lea como una cadena de excusas, conviene recordar qué quedó comprobado con evidencia antes de la primera medición:

- los límites de los seis perfiles, leídos del cgroup, no solo declarados;
- que `memory.swap.max = 0` en los seis, así que no hay *swap* que enmascare la presión de memoria;
- que los dos motores contienen los mismos 500000 documentos, comparados en tres direcciones contra el archivo fuente;
- que las dos consultas devuelven el mismo resultado que una referencia calculada en Python con aritmética exacta;
- que el instrumento *sabe* ver lo que afirma medir: con sondas dedicadas se forzaron throttling, volcado en los dos motores y OOM, y se comprobó que los mismos detectores y el mismo clasificador del runner los registran. Esas sondas viven aparte y **no producen filas del CSV**: mezclarlas con los resultados sería fabricar evidencia;
- que las dos fuentes de telemetría miden lo mismo (el `rss` de cAdvisor es exactamente la memoria anónima del cgroup, comprobado con el contenedor congelado);
- una celda piloto completa con tres repeticiones, antes de gastar las horas de la matriz, con el criterio de dispersión fijado *de antemano*.

B.5 Reproducibilidad

Todo lo necesario para repetir el experimento está en el repositorio: el *compose* parametrizado por perfiles, el generador con su semilla, las consultas tal como se copiaron del manual, el runner, el recorrido de la matriz —reanudable— y las 108 filas con su archivo crudo por repetición. Las imágenes están fijadas por *digest*, no por etiqueta, de modo que un `docker pull` futuro no cambie las versiones medidas. El dataset de 500000 documentos no se versiona por tamaño, pero se regenera byte a byte con la semilla 42, y su huella SHA-256 está registrada.